

Technical Analysis

Solicitante / Créditos

CTO-Level Architecture Review
& Improvement Roadmap

DOCUMENT METADATA

Project: Plinius — Credit OS Platform

Page: /solicitante/credit

File: pliny/app/solicitante/credit/page.tsx

Date: April 27, 2026

Author: CTO Office — Architecture Review

Table of Contents

1. Executive Summary
2. Page Overview & Purpose
3. Architecture & Component Breakdown
4. Technology Stack
5. Data Model & Database Schema
6. Routing & Navigation
7. Authentication & Security
8. UI/UX Design System
9. State Management & Data Flow
10. Identified Issues & Technical Debt
11. Improvement Roadmap
12. Performance Considerations
13. Platform Security Audit: Admin Auth Vulnerability
14. Recommendations Summary

1. EXECUTIVE SUMMARY

The `/solicitante/creditos` page is a core component of the Plinius Credit OS platform. It serves as the borrower's (solicitante's) primary interface for monitoring their active credits, reviewing amortization schedules, and tracking payment progress. The page is built as a Next.js 16 client-side rendered component using Supabase as the backend-as-a-service for authentication and data persistence.

This page operates within a larger solicitante portal that includes 8 sibling routes covering solicitudes, financiamiento, ofertas, mensajes, verificación, datos, and a credit score dashboard. The architecture follows a single-page-per-route pattern with no shared component library, which presents both simplicity and technical debt implications.

Key Findings

- The page is fully functional and delivers a clear credit management experience for borrowers.
- Architecture is monolithic per-page: all components, styles, and utilities are co-located in a single 402-line file.
- Heavy use of inline styles (no Tailwind or CSS modules) creates maintenance burden and prevents design system consistency.
- All data types are 'any' — no TypeScript interfaces for credits or pagos, losing type safety benefits.
- No error handling on Supabase queries — silent failures could leave users with blank screens.
- No pagination, search, or filtering — will degrade with scale.

2. PAGE OVERVIEW & PURPOSE

What This Page Does

The créditos page is the borrower's credit portfolio dashboard. When a solicitante logs in and navigates to this route, they see:

- A KPI summary bar showing: active credits count, total outstanding balance, total original amount, and overdue count.
- A payment progress section showing visual progress bars for each active credit with saldo vs. monto original.
- A full credit table listing all credits (active, overdue, settled, restructured, written-off) with key metadata.
- A detail modal that opens on click, showing complete credit information including amortization schedule.

User Journey

1. User lands on the page !' auth check runs !' if unauthenticated, redirect to
2. Credits are fetched from the 'credits' table filtered by client_id = auth.user.id.
3. KPIs are computed client-side from the fetched data (no server aggregation).
4. User clicks a credit row !' modal opens !' pagos (payments) are fetched for
5. Amortization table renders inside the modal showing each payment's status, amounts, and dates.

Target User

The primary user is a 'solicitante' (borrower/applicant) — typically a Mexican SME or individual who has applied for credit through the Plinius platform. They use this page to: monitor debt obligations, understand upcoming payments, check if any credits are overdue, and review the full amortization breakdown of each credit.

3. ARCHITECTURE & COMPONENT BREAKDOWN

File Structure

```
pliny/app/solicitante/creditos/page.tsx (402 lines, single file)
```

Component Tree

The file defines 8 components, all co-located:

- `Ic` — Inline SVG icon helper. Renders a 16x16 SVG path with configurable size and color. Used throughout for inline icons without an icon library dependency.
- `fmt(n)` — Currency formatter for compact display (\$1.5M, \$250K). Uses Mexican locale (es-MX).
- `fmtFull(n)` — Full currency formatter with 2 decimal places for precise amounts.
- `fmtDate(d)` — Date formatter for long format (e.g., '27 abr 2026').
- `fmtDateShort(d)` — Short date formatter without year.
- `EstatusPill` — Visual status badge with colored dot, background, and border. Supports 5 states: activo, vencido, liquidado, castigado, reestructurado.
- `DPDBadge` — 'Days Past Due' indicator. Shows 'Al corriente' if 0, otherwise shows day count with color severity (>90 days = critical red, >30 = warning amber).
- `ProgressBar` — Horizontal bar showing payment completion percentage. Calculates (total - current) / total.
- `CreditDetail` — Modal component for individual credit view. Fetches pagos on mount, shows KPIs, progress, next payment alert, and full amortization table.
- `CreditosPage` — Main page export. Handles auth, fetches credits, renders header, KPI cards, progress section, credits table, and conditionally renders the modal.

Data Constants

Two configuration objects define visual metadata:

- `ESTATUS_META` — Maps 5 credit statuses to label/color/dot configurations for the `EstatusPill` component.
- `PAGO_META` — Maps 4 payment statuses (pagado, pendiente, vencido, parcial) to label/color pairs for the amortization table.

4. TECHNOLOGY STACK

Core Framework

Technology	Details
Next.js	v16.1.6 — App Router with 'use client' directive. This page is fully client-side
React	v19.2.3 — Latest stable. Uses hooks (useState, useEffect) exclusively.
TypeScript	v5.x — Used but with significant 'any' type escape hatches.

Backend & Data

Technology	Details
Supabase	v2.97.0 — PostgreSQL backend-as-a-service. Handles auth, database, and
Supabase SSR	v0.9.0 — Available but NOT used on this page (all queries are client-side).
Auth	Supabase Auth with session persistence, auto-refresh, and URL detection.

Styling

Technology	Details
Inline Styles	Primary approach — all component styles are written as React style objects.
<style> tag	A CSS string injected for animations (fadeUp, spin), hover effects, and shared
Geist Font	Google Fonts import — 'Geist' (sans) and 'Geist Mono' (monospace) from
Tailwind CSS	v4.x — Available in the project but NOT used on this page.

Other Project Dependencies (Not Used on This Page)

Package	Purpose
jspdf + autotable	PDF generation (used elsewhere in the platform, e.g., contracts/pagarés).
docx	Word document generation.
xlsx	Excel export capability.
zod	Schema validation (not applied on this page).
resend	Email delivery service.
lucide-react	Icon library — available but this page uses custom SVG paths instead.
@nodecdfdi/*	Mexican SAT (tax authority) integration for fiscal credential handling.
validate-rfc	RFC (Mexican tax ID) validation.
bcryptjs	Password hashing.

5. DATA MODEL & DATABASE SCHEMA

Credits Table (Supabase: 'credits')

The page queries the 'credits' table with `SELECT *` filtered by `client_id`. Based on field usage, the inferred schema is:

Column	Type / Usage
id	UUID — Primary key, also used as fallback display (first 8 chars).
client_id	UUID — Foreign key to <code>auth.users.id</code> . Filters credits per borrower.
folio	String — Human-readable credit identifier.
deudor	String — Debtor name / company name.
estatus	Enum — 'activo', 'vencido', 'liquidado', 'castigado', 'reestructurado'.
monto_original	Numeric — Original loan amount.
saldo_actual	Numeric — Current outstanding balance.
tasa_anual	Numeric — Annual interest rate (percentage).
plazo_meses	Integer — Loan term in months.
tipo_credito	String — Credit type description.
amortiza	String — Amortization type/frequency.
dpd	Integer — Days past due.
fecha_inicio	Date — Loan start date.
fecha_vencimiento	Date — Loan maturity date.
ultimo_pago	Date — Last payment received date.
garantia	String (nullable) — Collateral description.
notas	String (nullable) — Additional notes.
created_at	Timestamp — Record creation timestamp (used for ordering).

Pagos Table (Supabase: 'pagos')

Fetches in the CreditDetail modal, filtered by `credit_id`:

Column	Type / Usage
id	UUID — Primary key.
credit_id	UUID — Foreign key to <code>credits.id</code> .
numero_pago	Integer — Payment number in sequence.
fecha_vencimiento	Date — Payment due date.
capital	Numeric — Principal portion of payment.
interes	Numeric — Interest portion.
iva_interes	Numeric — VAT on interest (16% in Mexico).
monto_total	Numeric — Total payment amount.
status	Enum — 'pagado', 'pendiente', 'vencido', 'parcial'.

6. ROUTING & NAVIGATION

Current Page Route

```
/solicitante/creditos !' pliny / app / solicitante / credit
```

Parent Layout

This page is wrapped by `pliny/app/solicitante/layout.tsx` which provides:

- A collapsible dark sidebar (240px open / 64px closed) with Plinius branding.
- Navigation to all 8 solicitante sub-routes.
- Authentication guard — redirects to `/login` if unauthenticated.
- Onboarding guard — redirects to `/onboarding/solicitante` if onboarding not complete.
- Real-time unread message badge via Supabase Realtime channels.
- Pending offers badge count.
- Company profile display and logout functionality.

Sibling Routes (Solicitante Portal)

Route	Purpose
<code>/solicitante</code>	Dashboard home — overview page.
<code>/solicitante/solicitudes</code>	Credit applications management.
<code>/solicitante/financiamiento</code>	Financing options and marketplace.
<code>/solicitante/ofertas</code>	Offers received from lenders (with pending badge).
<code>/solicitante/creditos</code>	THIS PAGE — Credit portfolio & amortization.
<code>/solicitante/mensajes</code>	Messaging with lenders (with unread badge).
<code>/solicitante/verificacion</code>	Identity/business verification.
<code>/solicitante/datos</code>	Personal/company data management.
<code>/solicitante/score</code>	Credit score dashboard.

Related Platform Routes

Route Area	Purpose
<code>/dashboard/*</code>	Lender (otorgante) portal — mirror functionality for the other side.
<code>/fondeador/*</code>	Funder portal — for investors funding credits.
<code>/admin/*</code>	Admin panel — superadmin, contracts, pagarés, verifications.
<code>/onboarding/*</code>	Multi-step onboarding for both borrowers and lenders.
<code>/login, /register</code>	Authentication flows.

7. AUTHENTICATION & SECURITY

Authentication Flow

The page implements a client-side auth guard:

```
const { data: auth } = await supabase.auth.getUser();  
if (!auth.user) { router.push('/login'); return; }
```

This runs inside a `useEffect` on mount. The parent layout (`SolicitanteLayout`) also has its own auth guard, creating a double-check. However, both are client-side, meaning:

- The page briefly renders before the auth check completes (flash of loading state).
- No server-side protection — the page HTML is delivered regardless of auth status.
- Row-Level Security (RLS) on Supabase should be the actual security boundary, not the client-side redirect.

Data Access Control

Credits are filtered by `client_id = auth.user.id`, which is correct. However:

- If Supabase RLS policies are not properly configured, a malicious client could bypass the `client_id` filter and access other users' credits by modifying the query.
- The `pagos` query in `CreditDetail` filters by `credit_id` but does NOT verify that the credit belongs to the current user — this is a potential IDOR vulnerability if RLS is not enforced.

Security Recommendations

- **CRITICAL:** Verify that Supabase RLS policies on 'credits' and 'pagos' tables enforce user ownership.
- Move auth checks to middleware or server components to prevent unauthorized page loads.
- Avoid `SELECT *` — only request columns that the UI actually uses.

Typography

- Primary font: 'Geist' (Vercel's font) — loaded via Google Fonts CDN.
- Monospace: 'Geist Mono' — used for numeric data, codes, and labels.
- Weight scale: 300-900, heavily using 700 (bold) and 800 (extra-bold).
- Letter-spacing: negative tracking (-0.03em to -0.04em) for headings, positive tracking for labels.

Color Palette

Token	Hex / Usage
Primary	#059669 (Emerald 600) — progress bars, active states, positive indicators.
Text Primary	#0F172A (Slate 900) — main body text and headings.
Text Secondary	#94A3B8 (Slate 400) — labels, subtitles, metadata.
Text Tertiary	#64748B (Slate 500) — mid-priority information.
Success	#10B981 (Emerald 500) — 'al corriente', zero balance, active dot.
Warning	#F59E0B / #92400E — amber tones for pending and upcoming payments.
Danger	#F43F5E / #9F1239 — red/rose for overdue, vencido status.
Card BG	#F8FAFC (Slate 50) — card interior backgrounds.
Border	#E8EDF5 — card and section borders.

Animation

- fadeUp — Entry animation with 8px translateY and opacity. 350ms with cubic-bezier(.16,1,.3,1).
- Staggered delays: .d1 (50ms), .d2 (100ms), .d3 (150ms) for cascading card entrances.
- spin — Loading spinner rotation at 700ms linear.
- Hover transitions: 120ms background change on credit rows.

Layout Patterns

- CSS Grid for KPI cards (4 columns) and amortization table (7 columns).
- Fixed-position modal with backdrop blur (4px) and dark overlay (45% opacity).
- 85vh max-height modal with internal scroll for amortization table (260px max).
- Responsive concerns: Grid columns are fixed — no media queries exist. This will break on mobile.

9. STATE MANAGEMENT & DATA FLOW

Component State (CreditosPage)

State Variable	Type / Purpose
credits	any[] — Full list of user's credits from Supabase.
loading	boolean — Controls skeleton/spinner display during fetch.
selected	any null — Currently selected credit for modal display.

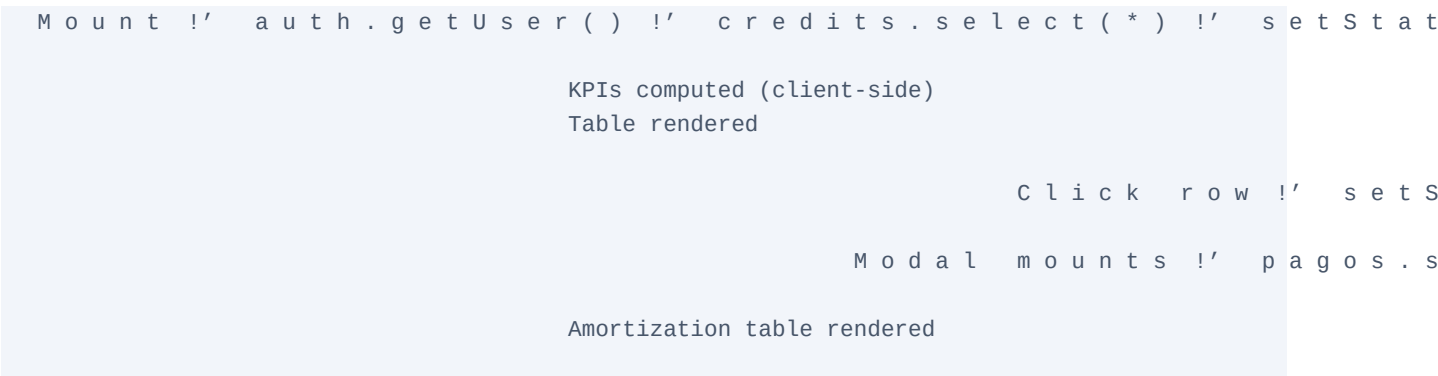
Component State (CreditDetail Modal)

State Variable	Type / Purpose
pagos	any[] — Payment schedule for the selected credit.
loading	boolean — Controls loading state inside the modal.

Derived State (Computed on Every Render)

Variable	Computation
activos	credits.filter(c => c.estatus === 'activo')
totalSaldo	Sum of saldo_actual for active credits.
totalOrig	Sum of monto_original for active credits.
vencidos	Count of credits with estatus === 'vencido'.
pagados (modal)	Count of pagos with status === 'pagado'.
proxPago (modal)	First pago with status 'pendiente' or 'vencido'.

Data Flow Diagram



10. IDENTIFIED ISSUES & TECHNICAL DEBT

P0 — Critical

- No error handling on Supabase queries: If the credits query fails, the page silently shows an empty state with no error feedback. The user has no way to distinguish 'no credits' from 'query failed'. Same issue in the CreditDetail modal for pagos.
- Potential IDOR on pagos: The CreditDetail component fetches pagos by credit_id without verifying that the credit belongs to the authenticated user. If RLS is not configured, any user could view another user's amortization schedule by manipulating the credit object.
- No TypeScript types: All data is typed as 'any'. This means no compile-time validation, no autocomplete, and no refactoring safety. A typo in a field name (e.g., 'estatus' vs 'status') would silently fail at runtime.

P1 — High Priority

- No pagination or virtual scrolling: All credits are fetched with SELECT * in a single query. For a user with 100+ credits, this will cause slow load times and high memory usage.
- SELECT * antipattern: Fetching all columns when only ~15 are used wastes bandwidth and exposes potentially sensitive data to the client.
- Inline styles everywhere: ~200 style objects are defined inline, making the code difficult to read, impossible to theme, and preventing hover/focus/media-query styling without the <style> tag workaround.
- No responsive design: The 4-column KPI grid and 7-column amortization table have fixed widths. On screens below ~900px, content will overflow or compress illegibly.
- Double auth guard: Both the layout and the page run supabase.auth.getUser() independently. This is redundant and doubles the auth request on page load.

P2 — Medium Priority

- Stale data: Credits are fetched once on mount. If a payment is made in another tab or by an admin, the page won't reflect the change until refresh. No Supabase Realtime subscription exists for credits/pagos (unlike mensajes in the layout).
- No loading skeleton: The KPI cards show a gray rectangle during load, but it's not a proper skeleton. The progress section and table have better loading states but could benefit from content-aware skeletons.
- Modal not keyboard-accessible: No ESC key handler, no focus trap, no aria attributes. Screen readers cannot identify the modal or its purpose.
- Custom SVG icons instead of lucide-react: The project has lucide-react installed (v0.576.0) but this page creates custom SVG paths via the Ic component. This is inconsistent with other parts of the platform that may use lucide.

11. IMPROVEMENT ROADMAP

Phase 1 — Stability & Security (1-2 Weeks)

- Add TypeScript interfaces for Credit and Pago types. Replace all 'any' with proper types.
 - Define: interface Credit { id: string; folio: string; deudor: string; estatus: CreditStatus; ... }
 - Define: type CreditStatus = 'activo' | 'vencido' | 'liquidado' | 'castigado' | 'reestructurado';
- Implement error handling with user-facing error states:
 - Show error banner when query fails with retry button.
 - Distinguish empty state from error state.
- Audit and enforce Supabase RLS policies on 'credits' and 'pagos' tables.
- Replace SELECT * with explicit column lists.
- Add ESC key handler and focus trap to the CreditDetail modal.

Phase 2 — UX & Performance (2-4 Weeks)

- Migrate inline styles to Tailwind CSS classes (already installed in the project).
- Add responsive breakpoints: stack KPI cards to 2x2 on tablet, 1-column on mobile.
- Implement pagination or infinite scroll for the credits table.
- Add search/filter by estatus, folio, deudor, date range.
- Add Supabase Realtime subscription for credit status changes and payment updates.
- Extract reusable components (EstatusPill, ProgressBar, DPDBadge) to a shared component library.

Phase 3 — Features & Scale (1-2 Months)

- Server-side rendering: Move the initial data fetch to a server component for faster first paint and SEO.
- Add export functionality: PDF statement per credit, Excel export of amortization table (jspdf/xlsx already in deps).
- Payment action integration: Allow solicitantes to initiate or confirm payments from the detail modal.
- Push notifications: Notify when a payment is approaching due date or becomes overdue.
- Analytics dashboard: Historical trends, payment patterns, credit utilization charts.
- Consolidate auth to middleware using Next.js middleware.ts + Supabase SSR.

12. PERFORMANCE CONSIDERATIONS

Current Performance Profile

The page has two data-fetching waterfalls:

```
Waterfall 1 (on mount):  
  a u t h . g e t U s e r ( )    ! '    c r e d i t s . s e l e c t ( * )    ! '    r e n d e r  
  ~200ms                      ~300-500ms                      ~50ms  
  
Waterfall 2 (on modal open):  
  p a g o s . s e l e c t ( * )    ! '    r e n d e r  
  ~200-400ms                      ~30ms
```

- Total time to interactive: ~550-750ms for the main page, plus ~200-400ms additional for each modal open.
- No caching: Every mount triggers a fresh Supabase query. React Query or SWR could cache and deduplicate.
- No prefetching: When hovering over a credit row, pagos could be prefetched to make modal opening instant.
- Google Fonts: External font loading adds ~100-300ms on cold cache. Consider using next/font for self-hosting.

Bundle Impact

- The `<style>` tag with `@import` for Google Fonts blocks rendering until the font CSS is loaded. This is a render-blocking resource.
- The custom `Ic SVG` component is lightweight (~3 lines) but creates new SVG elements on every render. For static icons, these could be memoized.
- No code splitting beyond Next.js defaults. The modal component (`CreditDetail`) could be lazy-loaded with `React.lazy()` since it's not needed until a user clicks.

Scalability Concerns

- At 1,000+ credits: The client-side filter/reduce for KPIs would process all records on every render. Move aggregation to a Supabase view or RPC function.
- At 100+ pagos per credit: The 260px scrollable amortization table handles this reasonably, but virtual scrolling would be beneficial for very long-term credits (e.g., 360 months).

13. PLATFORM SECURITY AUDIT: ADMIN AUTH VULNERABILITY

CRITICAL — Shared Auth Surface Between Admin and User Login

During the analysis of the `/solicitante/creditos` page's auth flow, a platform-wide security vulnerability was identified in how admin authentication is structured relative to user authentication. Both `/login` (users) and `/admin/login` (admins) authenticate against the same Supabase auth instance, the same `auth.users` table, and the same anon key. The differentiation between admin and user roles is enforced entirely on the client side.

Current Architecture

```
User Login (/login)      Admin Login (/admin/login)
  \                      /
   \                    /
    !'  Same supabaseClient.ts !•
    !'  Same auth.users table !•
    !'  Same anon key !•
    |
  localStorage 'bcl_session'
  { role: 'client' | 'super_admin' }
```

What Admin Login Does Right

- Checks the 'super_admins' table after Supabase auth to verify the user has admin access.
- Requires TOTP MFA (2FA via authenticator app) — enrolled via Supabase MFA API.
- Signs out the user immediately if they are not found in the super_admins table.

Critical Vulnerabilities Identified

V1: localStorage-Based Authorization (Trivially Bypassable)

Session role is stored in localStorage key 'bcl_session'. Functions like `requireSuperAdminSession()` simply read this value. Any user can open DevTools, set role to 'super_admin', and bypass all client-side route guards for `/admin/*`.

V2: No Server-Side Route Protection (No middleware.ts)

The project has no Next.js `middleware.ts` file. The `/admin` page HTML, JavaScript bundle, and component code are served to every visitor regardless of authentication status. Supabase SSR package is installed (v0.9.0) but unused.

V3: Admin Emails Hardcoded in Client Bundle

`lib/auth.ts` exports `SUPER_ADMIN_EMAILS` and `ADMIN_EMAILS` as Sets containing real email addresses (e.g., `luis@crowdlink.mx`, `jero@crowdlink.mx`). These are compiled into the client-side JS bundle and visible to anyone inspecting the source.

V4: Cross-Login Probing

A regular user who logged in via `/login` can navigate to `/admin/login`, re-enter the same credentials, and discover whether they exist in the `super_admins` table via the error message 'No tienes acceso al panel de administracion'. This leaks role membership.

Attack Scenario — Full Chain

1. Attacker logs in as regular user via /login
2. Opens DevTools !' Console :
localStorage.setItem('bcl_session',
JSON.stringify({role:'super_admin',email:'x@y.com'}))
3. Navigates to /admin
4. All client-side guards pass (requireSuperAdminSession reads localStorage)
5. Admin panel renders — attacker sees admin UI
6. ONLY barrier: Supabase RLS policies on the DB queries the admin panel makes (if configured correctly)

Affected Files

File	Issue
lib/auth.ts	Session stored in localStorage; admin emails hardcoded in client bundle.
app/admin/login/AdminLoginClient.tsx	Auth + super_admins check is good, but bypassable via localStorage.
app/admin/SuperAdminClient.tsx	Admin panel with no server-side protection.
app/login/page.tsx	User login uses same Supabase client; role stored in localStorage.
(missing) middleware.ts	Does not exist — no server-side route guards.

Recommended Fix — Priority Order

- P0 — Create middleware.ts: Use Next.js middleware with Supabase SSR to validate auth server-side for /admin/* routes. Redirect unauthenticated or non-admin users before the page JS is even delivered.

```
// middleware.ts (simplified)
import { createServerClient } from '@supabase/ssr'
import { NextResponse } from 'next/server'

export async function middleware(request) {
  const supabase = createServerClient(...)
  const { data: { user } } = await supabase.auth.getUser()
  if (request.nextUrl.pathname.startsWith('/admin')) {
    if (!user) return NextResponse.redirect('/login')
    const { data: sa } = await supabase
      .from('super_admins').select('user_id')
      .eq('user_id', user.id).maybeSingle()
    if (!sa) return NextResponse.redirect('/login')
  }
}
```

- P0 — Move admin session validation to server-side: Never trust localStorage for authorization. Use httpOnly cookies or Supabase SSR session tokens that cannot be manipulated client-side.
- P0 — Remove hardcoded admin emails from client bundle: Move ADMIN_EMAILS and SUPER_ADMIN_EMAILS to server-only code (e.g., env vars or DB-only checks). Mark the file with 'server-only' import.
- P1 — Separate admin auth surface: Consider using a different Supabase project for admin, or at minimum enforce strict RLS so admin panel queries fail without server-validated admin session.
- P1 — Add rate limiting on /admin/login: Prevent credential stuffing and brute-force attacks against the admin endpoint.

- P2 — Separate deployment: Deploy the admin panel on a different subdomain (e.g., admin.plinius.mx) with its own auth flow, making reconnaissance harder and blast radius smaller.
- P2 — Generic error messages: Change 'No tienes acceso al panel de administracion' to a generic 'Credenciales invalidas' to prevent role membership enumeration.

14. RECOMMENDATIONS SUMMARY

Priority Matrix

P0	Create middleware.ts for /admin/* server-side auth	1-2 days	Blocks localStorage bypass attack
P0	Move admin session to server-side (httpOnly cookies)	1-2 days	Eliminates client-side role spoofing
P0	Remove hardcoded admin emails from client bundle	0.5 days	Prevents admin identity
P0	Add TypeScript interfaces for all data types	1-2 days	Type safety, refactor confidence
P0	Add error handling on all Supabase queries	1 day	User trust, debuggability
P0	Audit RLS policies on credits/pagos tables	0.5 days	Data security, IDOR prevention
P1	Separate admin auth surface from user auth	2-3 days	Defense in depth, blast radius
P1	Add rate limiting on /admin/login	1 day	Prevents brute-force attacks
P1	Replace SELECT * with explicit columns	0.5 days	Performance, security
P1	Add responsive design / mobile support	2-3 days	Mobile user experience
P1	Migrate to Tailwind CSS (project already uses it)	2-3 days	Consistency, maintainability
P1	Add pagination to credits table	1-2 days	Scalability
P1	Deduplicate auth guard (layout already handles it)	0.5 days	Performance, clean code
P2	Use generic error messages on admin login	0.5 days	Prevents role membership leaking
P2	Consider separate admin subdomain deployment	3-5 days	Isolation, reconnaissance defense
P2	Add keyboard accessibility to modal	1 day	Accessibility compliance
P2	Implement Realtime subscriptions for live updates	1-2 days	Data freshness
P2	Extract shared components to component library	2-3 days	Reusability across portal
P2	Add search/filter capabilities	2-3 days	User productivity
P3	Server-side rendering migration	3-5 days	Performance, SEO
P3	Lazy-load CreditDetail modal	0.5 days	Bundle size
P3	Use next/font instead of Google Fonts @import	0.5 days	Performance

Final Assessment

The /solicitante/creditos page is a solid, functional credit dashboard that serves its core purpose well. The UI design is polished and the data presentation is thoughtful — the combination of KPI cards, progress bars, and detail modals creates a good user experience for borrowers monitoring their credit portfolio.

However, the analysis uncovered two categories of urgent work. First, the page-level technical debt: missing TypeScript types, absent error handling, and no responsive design. Second — and more critically — a platform-wide security vulnerability where admin authentication relies on client-side localStorage role checks with no server-side middleware. This means any authenticated user can escalate to admin privileges by editing localStorage.

The recommended approach is: (1) Immediately create middleware.ts to protect /admin/* routes server-side and remove hardcoded admin emails from the client bundle — this is a 1-2 day effort that closes the most dangerous attack vector. (2) Address the page-level P0 items (types, error handling, RLS audit) within the same sprint. (3) Work through P1 and P2 items systematically over the following 2-4 weeks as the platform matures.